Table 6: Ablation study results (GPT-5.2 thinking). Base: raw shell access with reactive prompting. Each row adds a component cumulatively.

| Configuration | XBOW | Pentest-Ben | GOAD |
|---|---|---|---|
| Base | 54 | 8 | 2 |
| + Tool Layer | 68 | 9 | 2 |
| + TDA-EGATS | 77 | 11 | 3 |
| + Memory (Full) | **85** | **12** | **4** |

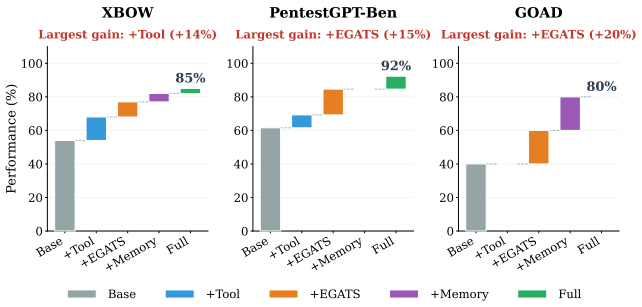


Figure 3: Ablation study across benchmarks (GPT-5.2 thinking). Performance is normalized to percentage scale.

while baselines gain only 1 machine each but plateau at 9.

GOAD shows the largest improvement. PENTESTGPT V2 compromises 4 of 5 hosts with GPT-5.2 and Opus 4.5 thinking (4 hosts in all three trials; the same four hosts each time) versus at most 2 for baselines—doubling the compromise rate (80% vs. 40%). This pattern holds consistently across all three models and both reasoning modes (even Gemini 3 achieves 3 hosts vs. 1–2 for baselines), indicating a robust architectural effect. Baselines achieve initial foothold but fail to progress through lateral movement; PENTESTGPT V2 executes coherent multi-host attack chains using the Memory Subsystem for credential persistence and TDA for exploration guidance.

## 5.3 RQ2: Ablation Study

To isolate each component's contribution, we evaluate system variants with individual components disabled. Table 6 presents results using GPT-5.2 thinking mode; the base configuration uses raw shell access with reactive prompting and sliding-window context management. Figure 3 visualizes component contributions across all model configurations.

The results align with our Type A/B failure framework. The Tool Layer provides the largest improvement on XBOW (+14 points, from 54 to 68), consistent with CTF failures being predominantly engineering problems addressable through better tooling. The Tool Layer alone yields zero improvement on GOAD (remaining at 2 hosts), where planning rather than capability determines success.

Table 7: Search behavior comparison on the PentestGPT benchmark (mean across 13 machines).

| Metric | PentestGPT | PENTESTGPT V2 |
|---|---|---|
| Branches explored | 3.2 | 7.8 |
| Backtrack rate (%) | 8 | 34 |
| Avg. depth before pivot | 12.4 | 5.1 |
| Successful pivots | 0.4 | 2.6 |
| Pruned branches | – | 4.2 |

TDA-EGATS adds further gains: +9 points on XBOW (from 68 to 77) through reduced trial-and-error, +2 machines on the PentestGPT benchmark (from 9 to 11), and +1 host on GOAD (from 2 to 3). These gains span both Type A failures (via more efficient search) and Type B failures (via principled exploration-exploitation). The Memory Subsystem contributes across all benchmarks: +8 points on XBOW (from 77 to 85), +1 machine on the PentestGPT benchmark (from 11 to 12), and +1 host on GOAD (from 3 to 4). The GOAD improvement is worth noting separately: extended attack campaigns cause context forgetting in systems without explicit state management, and Memory enables the credential persistence required for the fourth compromise.

## 5.4 RQ3: Strategy Analysis

Beyond aggregate performance, we analyze how TDA-EGATS changes the agent's attack strategy compared to PentestGPT's PTT-based approach.

### 5.4.1 Search Behavior

Table 7 compares exploration patterns across the PentestGPT benchmark. The metrics show qualitatively different search behaviors between the two systems.

PentestGPT follows a deep-first pattern: it explores fewer branches (3.2 vs. 7.8) but commits to each for longer (average depth 12.4 steps before pivoting vs. 5.1 for PENTESTGPT V2), reflecting the premature commitment failure mode where agents persist on initial hypotheses without signals to recognize intractability.

PENTESTGPT V2 with TDA-EGATS follows an adaptive pattern: TDI monitoring triggers backtracking when success rate drops, and evidence confidence guides exploitation timing. The 4.2 pruned branches per machine are paths abandoned due to persistently high TDI, preventing the infinite loops observed in baseline systems.

### 5.4.2 Case Study: HTB Falafel

Falafel is a Hard-rated HTB machine requiring a multi-stage attack chain that combines web exploitation, cryptographic

quirks, and privilege escalation through Linux group memberships. Figure 4 contrasts how PentestGPT and PENTESTGPT V2 navigate this challenge.

The attack begins with web enumeration revealing a login form that produces different error messages for valid versus invalid usernames, enabling user discovery through fuzzing. Boolean-based blind SQL injection in the username field allows extracting password hashes from the database. The key step is recognizing that the admin hash begins with "0e462...", a format that PHP's loose comparison operator (==) interprets as scientific notation. Submitting the string "240610708" produces an MD5 hash also starting with "0e", causing both values to compare as zero and bypassing authentication without password cracking. Post-authentication, a filename truncation vulnerability enables code execution: the system truncates filenames exceeding 237 characters, so uploading a file named `[232 A's].php.png` results in an executable `.php` file after truncation removes the `.png` extension. Privilege escalation chains through three stages: database credentials in the PHP configuration yield user `moshe`; membership in the `video` group enables framebuffer capture that reveals `yossi`'s password displayed on screen; membership in the `disk` group allows reading root's files directly via `debugfs`.

PentestGPT successfully extracts the password hashes but commits to direct cracking via hashcat. After 47 failed attempts with various wordlists and rules, context degradation prevents the model from revisiting the hash format—the type juggling vector is never considered.

PENTESTGPT V2's EGATS tree develops differently. When hash cracking yields repeated failures, rising TDI triggers exploration of authentication alternatives. The Knowledge Augmentation component surfaces PHP type juggling documentation when queried about hashes starting with "0e", enabling the bypass. The Memory Subsystem preserves credentials discovered at each privilege escalation stage, enabling the complete chain from `www-data` through `moshe` and `yossi` to `root`.

### 5.4.3 Failure Case: PlayerTwo

To illustrate where TDA-EGATS falls short, we examine PlayerTwo, the only PentestGPT Benchmark machine PENTESTGPT V2 fails to compromise. PlayerTwo requires exploiting a custom Protobuf-based game protocol with no public documentation. PENTESTGPT V2 correctly identifies the service through reconnaissance and spawns hypothesis branches for protocol fuzzing. However, TDI rises rapidly due to repeated failures (low $S$) and high horizon estimates (the LLM cannot predict steps for an unknown protocol). After three unsuccessful fuzzing attempts, the branch is pruned correctly by TDA's design logic, since success rate indicates intractability.

This failure exposes a TDA limitation: it cannot distinguish "difficult but tractable" from "novel requiring creative reasoning," as both present as high TDI. When RAG retrieval finds no relevant documentation and the LLM lacks parametric knowledge, TDA's evidence-based signals provide no useful guidance. TDA-EGATS therefore improves navigation through *known* attack spaces but does not address *novel* exploitation requiring genuine invention.

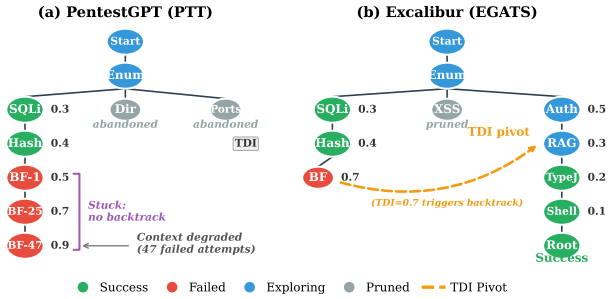


Figure 4: HTB Falafel exploration comparison. (a) PentestGPT commits to password brute-force after extracting hashes and stalls after 47 attempts. (b) PENTESTGPT V2's TDI-guided exploration discovers the type juggling bypass when hash cracking fails, then navigates the privilege escalation chain.

Table 8: Resource consumption per task (median values, GPT-5.2 thinking).

| Benchmark | LLM Calls | Time (min) | Cost ($) |
|---|---|---|---|
| XBOW | 12 | 3.2 | 0.18 |
| PentestGPT-Ben | 87 | 42 | 4.20 |
| GOAD | 234 | 186 | 28.50 |

## 5.5 RQ4: Real-World Deployment

To assess practical viability, we evaluate PENTESTGPT V2's resource consumption. We further deploy it in a live competition environment to examine its real-world performance.

### 5.5.1 Cost Analysis

Table 8 presents the resource consumption across benchmarks. PENTESTGPT V2 requires 23% fewer LLM calls than the baseline average on XBOW (12 vs. 15.6 median calls per task) due to reduced trial-and-error from structured tool interfaces, while achieving 39% higher success rates (85% vs. 61%). On GOAD, total calls increase by 18% due to more thorough exploration enabled by EGATS, but this yields 2× more compromised hosts (4 vs. 2). On a per-success basis, PENTESTGPT V2 is 1.8× more cost-effective on XBOW and 1.7× more cost-effective on GOAD: the overhead of EGATS is more than offset by the higher success rates. A complete GOAD engagement costs approximately $28.50 and achieves 80% environment compromise (4 of 5 hosts), making automated penetration testing economically viable for enterprise

Table 9: HTB Season 8 performance by difficulty (May–August 2025). Total: 10/13 machines (76.9%).

| Difficulty | Completed | Total | Rate |
|---|---|---|---|
| Easy | 4 | 4 | 100% |
| Medium | 4 | 4 | 100% |
| Hard | 2 | 3 | 67% |
| Insane | 0 | 2 | 0% |
| **Total** | **10** | **13** | **76.9%** |

security assessments.

#### 5.5.2 Live Competition Deployment

We deployed PENTESTGPT V2 during HTB Season 8 (May–August 2025), a live competition with 13 newly released machines whose solutions remain unavailable until the season concludes. This provides a direct test of real-world viability: unlike retired benchmark machines, Season machines incorporate recent CVEs and novel attack chains with no public walkthroughs.

PENTESTGPT V2 with Opus 4.1 completed 10 of 13 machines (76.9%), achieving a global ranking in the top 100 out of 8,036 active participants.

Table 9 summarizes performance by difficulty. All four Easy machines and all four Medium machines were compromised successfully. Among Hard machines, PENTESTGPT V2 completed Certificate and RustyKey but failed on Mirage. Both Insane machines, *Sorcery* and *Cobblestone*, remained unsolved. The three failures, Mirage (Hard), Sorcery (Insane), and Cobblestone (Insane), represent machines where PENTESTGPT V2 exhausted its search space without finding viable attack paths. These results align with the PlayerTwo analysis (§5.4): when RAG retrieval yields no relevant documentation and the underlying model lacks parametric knowledge of the target vulnerability class, TDA-EGATS cannot guide exploration effectively.

The Season 8 deployment shows that PENTESTGPT V2 can operate in realistic penetration testing scenarios where solutions are unknown and time-constrained. The 100% success rate on Easy and Medium machines suggests readiness for deployment on typical enterprise targets, while Hard and Insane failures mark the current boundaries where human expertise is still required.

## 6 Discussion

### 6.1 Limitations and Threats to Validity

We discuss factors that bound the generalizability of our findings.

**Benchmark Scope.** Our evaluation covers web security, network penetration testing, and Active Directory attacks, but omits binary exploitation, mobile security, and cloud-specific attack scenarios where different challenges may dominate. Binary exploitation requiring precise memory layout reasoning poses distinct challenges not captured by our benchmarks. The PentestGPT Benchmark uses retired machines with public walkthroughs, which may inflate absolute numbers through data contamination; however, TDA, EGATS, and Memory target planning challenges orthogonal to specific vulnerability knowledge and thus transfer to novel scenarios. Real-world engagements also involve active defenses and novel vulnerability classes absent from historical benchmarks.

**Model-Specific Effects.** We obtain results with three frontier models (GPT-5.2, Claude-Opus-4.5, Gemini-3.0-Pro). Different model architectures show different strengths: Opus 4.5 achieves the highest XBOW performance (91%), which suggests that our architectural contributions may interact differently across model families. Future model generations may shift the easy/hard boundary and potentially resolve challenges we currently classify as hard.

**Baseline Fairness.** We use published baseline code with default parameters; original authors might achieve better results through tuning, though this reflects realistic deployment scenarios. Because baselines use their original tool invocation mechanisms, reported improvements reflect both tool integration and architectural contributions.

**Failure Analysis.** We analyze PENTESTGPT V2's remaining failures to characterize current boundaries. On XBOW, the 9 failed tasks (9%) fall into two categories: blind injection that requires extensive timing-based exfiltration (4 tasks), and multi-stage attacks that require creative payload chaining not present in our RAG corpus (5 tasks). The single unsolved PentestGPT Benchmark machine (PlayerTwo, Hard) requires exploiting a custom protocol with no public documentation, a novel exploitation scenario that demands reasoning beyond pattern matching. On GOAD, the fifth host (the forest root domain controller) requires a specific attack chain (PrintNightmare → DCSync) that PENTESTGPT V2 identifies but fails to execute due to token constraints. These failures indicate that while PENTESTGPT V2 addresses Type B failures effectively, novel exploitation that requires creative reasoning remains an open problem.

### 6.2 What Remains Hard

Despite PENTESTGPT V2's gains, three categories of irreducible Type B failures persist that better tooling, larger corpora, or improved prompting cannot resolve.

**The Creativity Barrier.** LLMs are effective at pattern matching but struggle with out-of-distribution generalization [21]. The PlayerTwo failure illustrates this gap: PENTESTGPT V2 systematically explores attack vectors yet fails because no documented exploitation pattern exists for the custom Protobuf-based protocol. The distinction between "difficult" and "novel" matters here. Difficult tasks respond to improved

search; novel tasks require reasoning capabilities that current architectures do not provide.

**The Adversarial Environment Barrier.** Penetration testing occurs against active defenders who can exploit agent reasoning patterns [33]. Honeypots, canary tokens, and deceptive services can poison the agent's state representation, causing it to pursue false attack paths or trigger detection. PENTESTGPT V2's evidence grounding protects against self-generated hallucinations but offers limited defense against environmentally-induced false beliefs: when a honeypot presents a convincing vulnerable service, the agent cannot tell whether the vulnerability is genuine or a deliberate trap. This asymmetry favors defenders, who can study and exploit agent behavior, while agents lack the meta-awareness to recognize manipulation.

**The Temporal Scale Barrier.** Human pentesters maintain mental models across engagements that span weeks, correlating information from separate sessions and exercising strategic patience. EGATS improves multi-step reasoning within sessions and the Memory Subsystem preserves state, but neither addresses cross-session continuity. Long-horizon planning is a different problem from long-context processing: it requires hierarchical abstraction, goal decomposition, and progress monitoring, none of which current transformer architectures natively support [27].

## 7 Conclusion

This paper presents a systematic analysis of LLM-based penetration testing that identifies a distinction between Type A failures (capability gaps addressable through engineering) and Type B failures (complexity barriers requiring architectural innovation). We introduce PENTESTGPT V2, which addresses Type A failures through a Tool and Skill Layer with typed interfaces and RAG, and addresses Type B failures via Task Difficulty Assessment (TDA) integrated into Evidence-Guided Attack Tree Search (EGATS). PENTESTGPT V2 achieves 91% task completion on CTF benchmarks (49% improvement over baselines) and compromises 4 of 5 hosts on the GOAD Active Directory environment versus 2 for prior systems. Our ablation studies show that TDA-guided exploration provides benefits beyond tree structure alone: difficulty-aware planning produces value that model improvements cannot replicate.

## References


[1] Vulnhub: Vulnerable by design. https://www.vulnhub.com/, 2012–2026.

[2] XBOW — AI-Powered Offensive Security Platform. https://xbow.com/, 2024.

[3] Anonymous. Excalibur: Source code and artifacts. https://anonymous.4open.science/r/Excalibur-FA7D, 2025. Anonymous repository for double-blind review.

[4] Anthropic. Equipping agents for the real world with Agent Skills, October 2024. Engineering Blog. URL: https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills.

[5] Anthropic. Model context protocol. https://modelcontextprotocol.io/, 2024. An open protocol for connecting AI assistants to external data sources and tools, released November 2024.

[6] Rémi Coulom. Efficient selectivity and backup operators in Monte-Carlo tree search. In *Computers and Games: 5th International Conference, CG 2006, Turin, Italy, May 29-31, 2006. Revised Papers 5*, pages 72–83. Springer, 2007. URL: https://link.springer.com/chapter/10.1007/978-3-540-75538-8_7, doi:10.1007/978-3-540-75538-8_7.

[7] Isaac David and Arthur Gervais. Multi-agent penetration testing ai for the web. *arXiv preprint arXiv:2508.20816*, 2025.

[8] Gelei Deng, Yi Liu, Víctor Mayoral-Vilches, Peng Liu, Yuekang Li, Yuan Xu, Tianwei Zhang, Yang Liu, Martin Pinzger, and Stefan Rass. PentestGPT: Evaluating and harnessing large language models for automated penetration testing. In *Proceedings of the 33rd USENIX Security Symposium (USENIX Security 24)*, pages 847–864. USENIX Association, 2024.

[9] Luca Gioacchini, Marco Mellia, Idilio Drago, Alexander Delsanto, Giuseppe Siracusano, and Roberto Bifulco. AutoPenBench: Benchmarking generative agents for penetration testing. *arXiv preprint arXiv:2410.03225*, 2024.

[10] Google Project Zero. From naptime to big sleep: Using large language models to catch vulnerabilities in real-world code. https://projectzero.google/2024/10/from-naptime-to-big-sleep.html, October 2024.

[11] Hack The Box. Hack the box: Hacking training for the best. https://www.hackthebox.com/, 2024. Online platform with curated collection of vulnerable machines for penetration testing practice and skill development.

[12] Andreas Happe and Jürgen Cito. Can LLMs hack enterprise networks? autonomous assumed breach penetration-testing active directory networks. *ACM Transactions on Software Engineering and Methodology*, 2025. doi:10.1145/3766895.

[13] Sean Heelan. How I used o3 to find CVE-2025-37899, a remote zeroday vulnerability in the Linux kernel's SMB implementation. https://sean.heelan.io/2025/05/22/how-i-used-o3-to-find-cve-2025-37899-a-remote-zeroday-vulnerability-in-the-linux-kernels-smb-implementation/, May 2025.

[14] ISC2. ISC2 cybersecurity workforce study 2024. https://www.isc2.org/Insights/2024/10/ISC2-2024-Cybersecurity-Workforce-Study, 2024.

[15] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. SWE-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*, 2024.

[16] Levente Kocsis and Csaba Szepesvári. Bandit based Monte-Carlo planning. In *Machine Learning: ECML 2006: 17th European Conference on Machine Learning, Berlin, Germany, September 18-22, 2006. Proceedings 17*, pages 282–293. Springer, 2006. URL: https://link.springer.com/chapter/10.1007/11871842_29, doi:10.1007/11871842_29.

[17] He Kong, Die Hu, Jingguo Ge, Liangxiong Li, Tong Li, and Bingzhen Wu. VulnBot: Autonomous penetration testing for a multi-agent collaborative framework. *arXiv preprint arXiv:2501.13411*, 2025.

[18] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024. doi:10.1162/tacl_a_00638.

[19] Phung Duc Luong, Le Tran Gia Bao, Nguyen Vu Khai Tam, Dong Huu Nguyen Khoa, Nguyen Huu Quyen, Van-Hau Pham, and Phan The Duy. xOffense: An AI-driven autonomous penetration testing framework with offensive knowledge-enhanced LLMs and multi agent systems. *arXiv preprint arXiv:2509.13021*, 2025.